

Audio Research Assistant

Detailed System Pipeline & Architecture

what we built · how each part works · how to improve it

A Python 3.11 retrieval-augmented research assistant. A FastAPI app streams answers as NDJSON while an orchestrator routes each question to the right engine: a local PDF RAG over Oracle 23ai vectors (hybrid vector + BM25 + rerank), an external multi-source web/paper search, a Corrective-RAG + agentic verify-rewrite answerer, and a fully autonomous code agent that writes, runs, and verifies Python in a locked-down Docker sandbox.

Contents

1. System Architecture (diagram)
2. Answering Pipeline, per request (diagram)
3. Autonomous Code Agent Loop (diagram)
4. Subsystems — what / how / strengths / improvements
 - PDF ingestion · hybrid retrieval · external search ·
 - answering (CRAG + agentic loop) · code agent ·
 - LLM provider layer · web app / API / frontend
5. Recent enhancements & improvement roadmap

1 · System Architecture

Local PDF RAG + external search + agentic answerer + autonomous code agent

User · Browser (vanilla-JS workspace)

question (NDJSON)

webapp/server.py — FastAPI · auth & sessions · /api/chat streams NDJSON events

webapp/chat_logic.stream_chat_events — Orchestrator: sanity → refine → classify → ROUTE

CODE INTENT

**backend/agent — autonomous
code agent (Docker sandbox)**

FOLLOW-UP

**conversation_router → answer
from chat / re-run prior code**

NEW RESEARCH

**deep-query plan → retrieve
(freshness → web-only + year)**

OFFLINE INGESTION
pipeline.py + backend/ingestion
PDF → parse (Docling) → chunk
→ embed → Oracle vector index

Local hybrid retrieval
vector + BM25 + RRF + rerank
backend/retrieval (Oracle 23ai)

External search
web · arXiv · S2 · Wiki ·
patents · GitHub

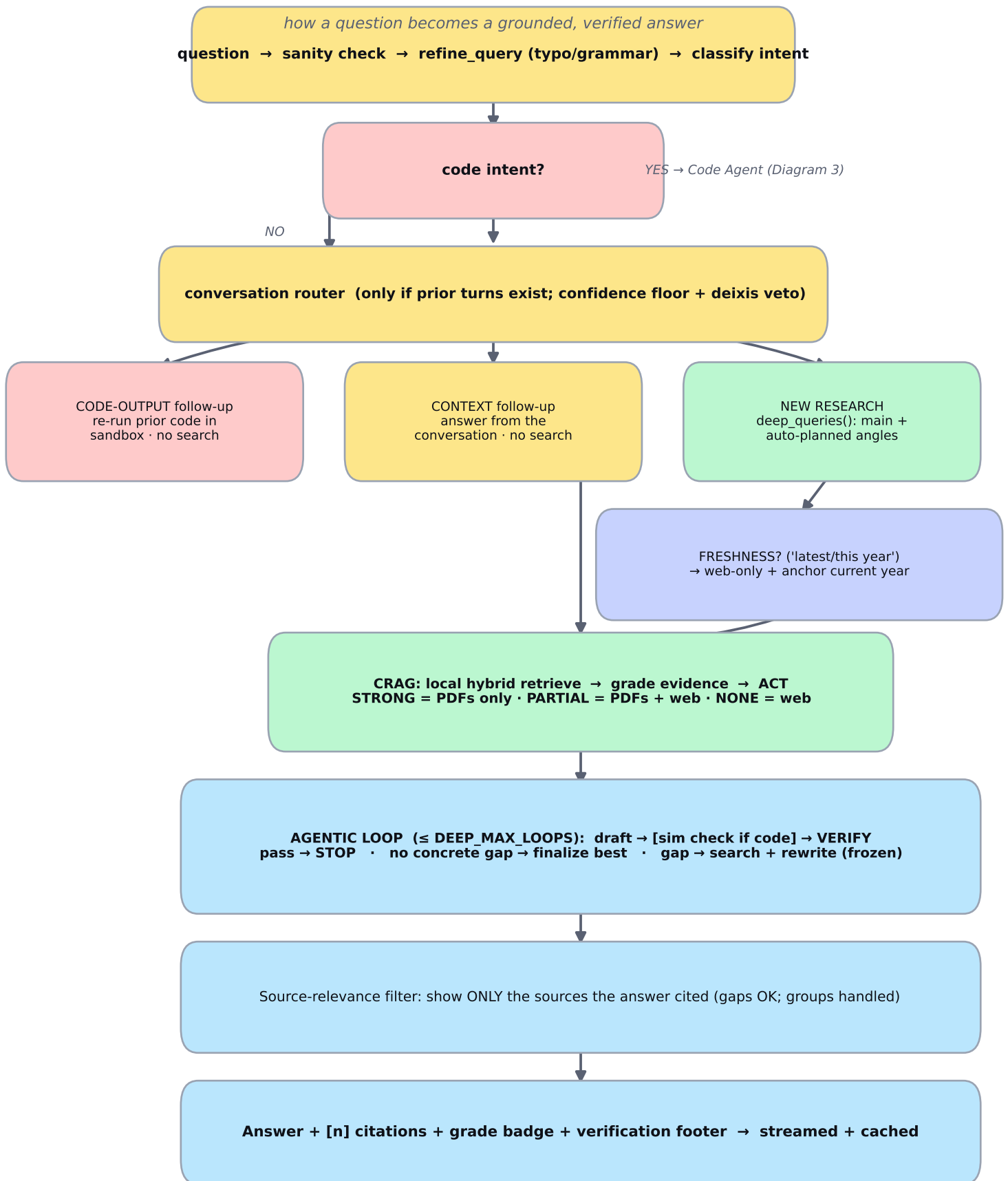
**CRAG grade (STRONG / PARTIAL / NONE) → Agentic verify→rewrite loop
backend/answering (early-stop · DEEP_MAX_LOOPS · Self-RAG)**

Answer + citations (only cited sources) + honest verification footer

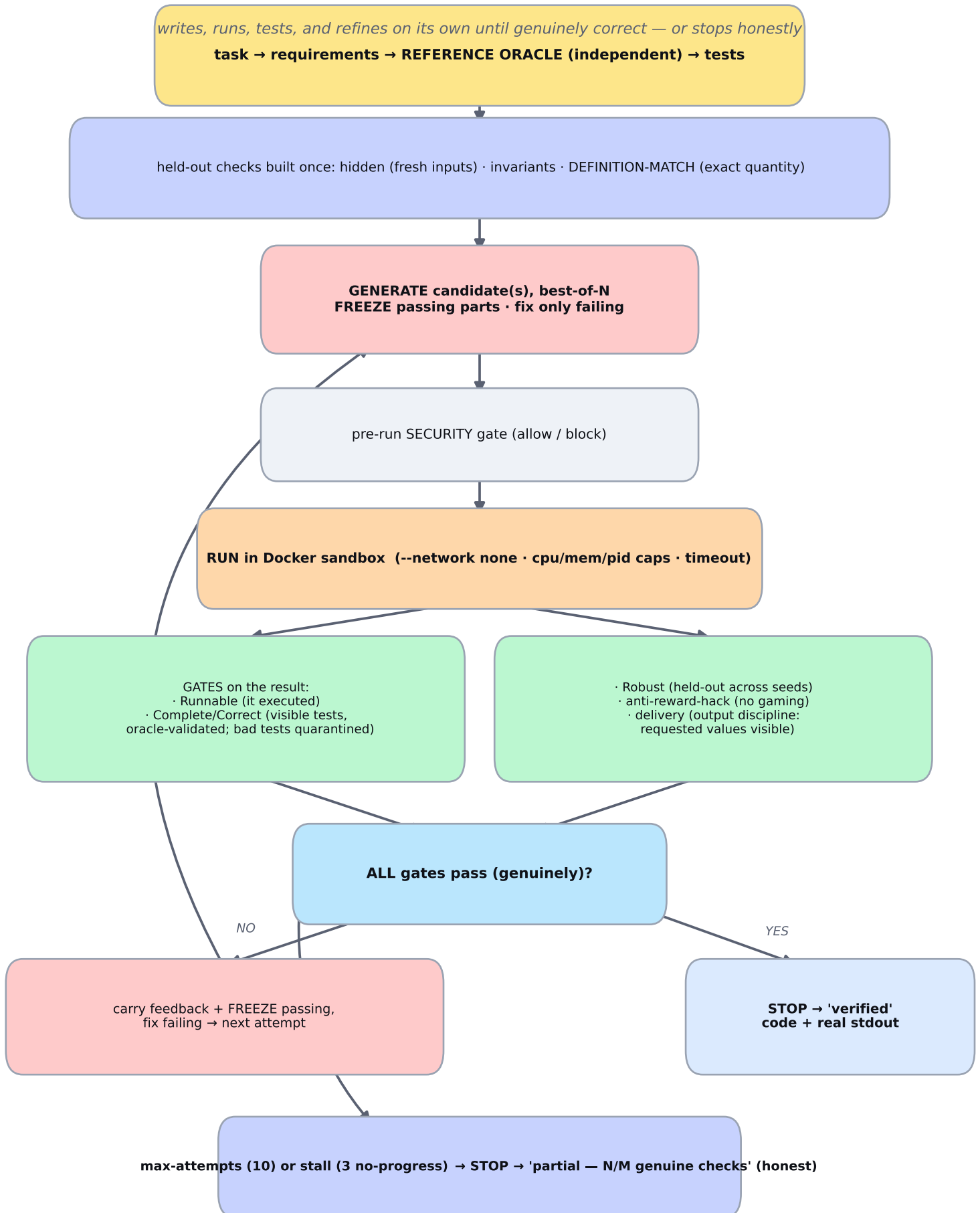
LLM provider layer — backend/llm: model routing · resilient fallback chain · 429 backoff · streaming

Every generative step (router, planner, drafter, verifier, code agent, ingest-embed) calls the LLM provider layer

2 · Answering Pipeline (per request)



3 · Autonomous Code Agent Loop



REAL failure → fix the CODE. FALSE failure (flawed test) → quarantine the TEST, never weaken a correct check.

4a · PDF Ingestion Pipeline (backend/ingestion, pipeline.py)

What it does

Turns the PDFs in data/papers/ into a searchable Oracle 23ai vector index. Per PDF it extracts per-page text (PyMuPDF, optionally enriched by the Docling layout/table parser and CPU OCR), splits into section-aware, type-tagged chunks, adds an Anthropic-style 'contextual retrieval' situating sentence per chunk, embeds each chunk to a 768-d vector (Gemini API or a local BGE model), and stores papers/chunks/native VECTORS.

How it works

pipeline.py runs each stage as a separate subprocess (a native crash in one can't kill the others) and stops on the first failure. Stage 1 (ingest_papers) hashes each PDF for dedup/idempotency, parses with PyMuPDF as the source of truth, and only runs heavy Docling when free memory exceeds DOCLING_MIN_FREE_MB (pre-empting native OOM); OCR runs only on text-poor pages. document_chunker packs sentences to ~1800 chars with 2-sentence overlap, extracts figure/table captions + algorithm blocks, tags section/type/concepts, and dedupes. Stage 2 (embed_chunks) embeds only rows WHERE embedding IS NULL (resumable after a 429), committing per batch. Stage 3 (vector_migration) converts the JSON embedding into a native VECTOR(768) column for cosine search. Incremental mode diffs a SHA-256 manifest and rebuilds only changed PDFs.

Strengths

- Crash isolation everywhere: per-stage subprocess, per-PDF try/except (incl. MemoryError), Docling free-memory guard + PyMuPDF fallback.
- Idempotent + resumable: content-hash dedup; embed only NULL rows with per-batch commits; incremental SHA-256 manifest with a no-op fast path.
- No silent page loss: explicit pages_indexed/pages_total coverage warnings.
- Retrieval-aware chunking: contextual situating sentence (cached, fail-safe), section/type tagging, caption + algorithm-block preservation.
- GPU discipline: parsing/OCR forced to CPU so the GPU stays free for the query-time reranker.

How to improve

- Write native VECTOR directly in Stage 2 (oracledb supports array binds)— removes Stage 3, the duplicate JSON-CLOB storage, and a per-row round-trip.
- Carry Docling per-item page numbers through to chunks: markdown-parsed papers currently get page_start=1/page_end=N, so their citations can't point to a real page.
- Act on 'deleted' PDFs in incremental mode — diff detects them but never purges the orphaned Oracle rows.
- Make chunking token-aware (tie CHUNK_MAX_CHARS to the embedder's token limit) instead of char-based, to avoid truncation drift across models.
- Stream embed_chunks with a paged/server-side cursor instead of fetchall() of all NULL rows (memory for very large libraries).
- vector_migration runs at import time with no main()/guard— wrap in functions so it's testable and importable without Oracle.

4b · Hybrid Local Retrieval (backend/retrieval)

What it does

Given a question, finds the most relevant chunks from the local corpus. Runs three retrievers in parallel—dense vector on the question, dense vector on a HyDE pseudo-answer, and field-weighted BM25—fuses them with Reciprocal Rank Fusion, cross-encoder reranks the pool, boosts intent-matching chunk types, and applies MMR diversification with a per-paper cap. (The `graph_reason` / GraphRAG field is plumbed to the UI but always empty.)

How it works

embed the query (Gemini/local) -> [vector_orig, optional vector_hyde, BM25F] computed concurrently on a thread pool -> RRF rank fusion (scale-robust: rank-based, not linear) -> take the top pool -> cross-encoder rerank vs the original question (fp16 on CUDA; warmup at startup) -> additive chunk-type boosts -> MMR with a hard `MAX_SOURCES_PER_PAPER` cap. Every stage degrades gracefully: cross-encoder OOM falls back to a 0..1 lexical reranker; turbovec cache failure falls back to Oracle; HyDE failure is non-fatal. FAST/DEEP knobs are read from env per request.

Strengths

- Genuinely hybrid (dense + sparse + HyDE) fused by rank-based RRF—robust to the cosine-vs-BM25 score-scale mismatch.
- Strong graceful degradation (a project hard rule): reranker/turbovec/HyDE all have tested fallbacks.
- Performance-conscious: three rankings concurrent; BM25 stats + chunk list cached; reranker fp16 + startup warmup.
- HyDE with no LLM/API call (template + lexicon)—recall boost at near-zero latency/cost.
- Research-corpus tuning: BM25F field weighting, contextual sentence indexed for BM25, intent-aware type boost, MMR per-paper cap.

How to improve

- BM25 is an $O(N)$ Python scan over every chunk per query—won't scale past a few thousand chunks; precompute per-field stats at index time or use an inverted index / Oracle Text.
- BM25 / chunk caches never invalidate in-process, so newly-ingested papers don't appear until restart; add a corpus-signature check (reuse turbovec's manifest pattern).
- FAST and DEEP share identical retrieval knobs—the distinction has no effect on local depth; differentiate them or document that mode only changes the external/answering layers.
- Normalize `rerank_score` before adding the fixed chunk-type/MMR boosts (cross-encoder vs lexical-fallback scores live on different scales).
- MMR similarity is word-level Jaccard—use cosine on the already-computed embeddings for better near-duplicate detection.
- Either implement the GraphRAG `graph_reason` field (`audio_concepts` is a natural seed) or remove the dead field.

4c · External Search Orchestration (backend/external_search)

What it does

An optional, opt-in evidence channel that supplements local RAG with the public internet. Fans a single query across web, arXiv, Semantic Scholar, Wikipedia, patents, and GitHub, reads online PDFs it finds, normalizes everything into uniform citation-bearing records, then de-dupes and re-ranks against the question. It never crashes the request: any failing/timed-out/keyless channel is skipped with a warning.

How it works

`clean_query()` -> fan out the selected channels concurrently on a thread pool (free: arXiv/Semantic Scholar/Wiki/GitHub; keys add web+patents) -> each channel hits its API through a shared cached, size-capped, backoff-aware `safe_get` -> results consumed via `as_completed` with a hard `EXTERNAL_GATHER_TIMEOUT` and `shutdown(wait=False)` so stragglers can't block -> content-hash de-dup -> `source_ranker` re-ranks (with a freshness bonus that scales up when the query says 'latest/recent/<year>'). Online PDFs are read sequentially and capped (`MAX_PDFS`) to bound memory.

Strengths

- Strong fault isolation: `gather_external_evidence` never raises; `safe_get` returns `None` on any error.
- True concurrency with a hard wall-clock cap (`as_completed` timeout + `shutdown(wait=False)`).
- Works with zero keys (arXiv/S2/Wiki/GitHub + keyless DuckDuckGo); keys only unlock quality/patents/code search.
- Uniform citation-first `ExternalSource` model with `content_hash()` de-dup and secret-stripping to `_public()`.
- Defense-in-depth fetching: 3 MB cap, timeout, streamed download, 429/5xx backoff, post-redirect re-validation, PDF magic-header check, TTL disk cache.
- Recency awareness in ranking so fresh papers surface without losing on-topic ones.

How to improve

- Move to async I/O (`httpx/aiohttp` + `asyncio.gather`) — cleaner cancellation than leaving daemon threads after `shutdown(wait=False)`.
- Add per-channel deadline budgets: the gather timeout only bounds the outer wait; a channel's own backoff + sequential PDF reads can still run long.
- Add near-duplicate detection (URL canonicalization + shingling/embedding)— exact content-hash misses arXiv abs/pdf/S2 duplicates of one paper.
- Harden the DuckDuckGo HTML scraper (a single point of failure); add a health metric and a second keyless fallback.
- Normalize cache keys and use per-channel TTLs (shorter for web/news, longer for arXiv abstracts).
- Re-introduce an opt-in SSRF/private-IP guard (`env-gated`, default off) so the subsystem is safe for untrusted/multi-tenant use.

Answering Pipeline — CRAG + Agentic Loop (webapp/chat_logic + backend/ans

What it does

The orchestration layer that turns a chat message into a grounded, cited answer streamed as JSON events. It decides HOW to answer (reuse cache, send to the code agent, answer a follow-up from the conversation, or run a fresh research sweep), then for research it retrieves evidence, grades local coverage (Corrective RAG), and draft-verify-rewrites in a bounded agentic loop, escalating a library-only answer to the web if it fails (Self-RAG).

How it works

sanity -> apply mode profile -> refine_query -> classify code-intent (route to code agent) -> conversation router (confidence floor + deixis veto: code-output follow-up re-runs prior code, context follow-up answers from chat, else research) -> answer-cache lookup -> freshness check (web-only + current-year anchor) -> deep_queries (main + auto-planned angles) -> CRAG: local hybrid retrieve, grade STRONG/PARTIAL/NONE, act (PDFs only / +web / web) -> agentic loop (<= DEEP_MAX_LOOPS): draft, optional sandbox sim-check for code, LLM verify; stop on pass or no-concrete-gap (one feedback-guided rewrite allowed), else search + rewrite keeping frozen parts -> auto-review -> repair_citations + show only cited sources -> stream + cache. Today's date is injected; an entity-attribution guard prevents crediting the wrong org.

Strengths

- Clean separation: each decision (intent, routing, grading, verification, review, citations) is a small testable module.
- Cost-aware: FAST/DEEP profiles, CRAG grade reuses reranker scores (zero extra LLM), answer/external/grade caches, evidence budgets, loop early-stop, bounded concurrency.
- Degrades gracefully and never breaks chat; freshness queries deliberately bypass caches and the stale local library.
- Strong grounding: LLM verifier + Self-RAG + auto-review + citation repair + cited-only source panel.
- Router safety net: LLM verdict AND a confidence floor AND a deixis veto— a confident-but-wrong 'follow-up' can't starve a new question of search.

How to improve

- Split stream_chat_events (~340 lines, deep nesting) into named helpers (agentic loop, CRAG act, finalize/cache) per the 50-line/4-level guideline.
- Reduce LLM fan-out: merge refine + classify + route into one structured call, or skip the router/classifier with no history / obvious prose.
- Make the mode profile request-scoped (contextvars/settings object) instead of mutating process-global os.environ, which can interleave across concurrent requests.
- Distinguish a verifier parse-failure (score defaults to 0) from a genuine low score— retry once / treat as inconclusive rather than firing a low-confidence banner.
- Include mode + source toggles in the answer-cache key so a FAST PDF-only answer isn't served for a later DEEP request.
- Add control-flow tests for the orchestrator branches (CRAG act, Self-RAG, early-stop vs feedback-rewrite, cache gating) with a fake provider + fake store.

4e · Autonomous Code Agent (backend/agent)

What it does

Given a coding/algorithm request, it writes Python, runs it in a locked-down throwaway Docker container, and only labels it 'verified' if it provably passes machine-checked correctness gates— never on 'it ran'. It generates its own acceptance criteria (requirements, visible tests, a reference oracle, held-out hidden tests, invariants, and one definition-match check per requested output), runs everything across random seeds, feeds genuine failures back, defends against reward-hacking, and labels honestly.

How it works

requirements -> task-type -> reference ORACLE (expected values by execution, not guessed) -> visible tests (oracle-validated; a test the oracle itself fails is quarantined) -> LOOP ($\leq$ AGENT_MAX_ATTEMPTS=10): generate best-of-N candidates that FREEZE already-passing parts and fix only failing ones -> pre-run security gate -> run in the sandbox (isolated _sol / ref modules so ref.* cheats hit NameError) -> gates: Runnable, Complete/Correct (visible), Robust (held-out across seeds), anti-cheat (static AST scan), delivery (output discipline) -> verified -> stop; else carry feedback + freeze, retry. Stops at max-attempts or stall (3 no-progress rounds) with an honest 'partial — N/M genuine checks'. REAL failure -> fix code; FALSE failure (flawed test) -> quarantine the test, never weaken a correct check.

Strengths

- Correctness is machine-verified across seeds: visible AND held-out AND invariants AND per-output definition-match — passing the visible tests is never sufficient.
- Reward-hacking defense: static AST scan + oracle isolation (candidate can't read the oracle) + behavioural held-out checks; a gamed attempt is never presented (rejected_cheating).
- Reference-oracle + test-quarantine: expected values by execution, and tests the known-correct oracle fails are removed so a flawed test can't fail correct code (definition checks stay oracle-independent).
- Honest labeling: verified / partial (exact N/M, stall-vs-cap reason) / rejected_cheating / failed— never a fake pass, never a prose 'this would output...'.
- Freeze-verified-parts + regression detection + bounded memory + stall detection = efficient, self-improving iteration.
- Sandbox: network-off, non-root, mem/CPU/PID caps, timeout, code on stdin, nothing mounted; output discipline keeps requested values visible (head+tail capture).

How to improve

- Generate the oracle/definition checks with a DIFFERENT provider than the solver to break correlated blind spots (same model family writes solution, oracle, and most checks).
- Tighten _check_completeness from token-substring to value-adjacent-to-label, so a printed-but-wrong number can't pass on the label word alone.
- Strengthen the relevance gate (_is_relevant_code is satisfiable by a comment/variable name); the optional LLM peer-review is the only stronger check.
- Early-exit a round once one candidate is fully verified (instead of running all N), and cache identical sandbox scripts, to cut up to $N \times (1 + \text{seeds})$ container runs.
- Skip the oracle/held-out machinery for trivially non-algorithmic tasks via a cheap complexity heuristic.
- Constrain build-time pip installs (ensure_image_for) to an allowlist/pinned index— the dep build has network, so a typosquat could execute at BUILD time.

4f · LLM Provider Layer (backend/llm)

What it does

A thin, provider-agnostic streaming chat layer: one factory (`get_provider`) and one method (`stream_chat`) so callers never care whether the model is Gemini, Mistral, OpenAI, OpenRouter, or local Ollama. It routes each model id to the right endpoint + key, adapts request params for quirky APIs, streams tokens (and optional hidden 'reasoning'), and shrinks the token budget on low-credit 402 errors. The resilient fallback chain lives one layer up (`ResilientProvider` in the agent).

How it works

`get_provider / route_model` resolve `model_id` -> (`base_url`, `api_key`) via `PROVIDERS/CATALOG` -> construct an OpenAI-compatible client -> `stream_chat` opens a `streaming chat.completions.create`, adapting token/temperature params per model (o-series `max_completion_tokens`, no-temperature models). On a 402 'can only afford N' it shrinks the budget and retries; if the stream can't open it re-raises the real error so the UI shows a true message. `ResilientProvider` composes this base across a model chain with `retry/backoff/fallback` (buffering each model's full reply to avoid duplicated output on fallback), respecting the user's explicit model choice.

Strengths

- Single OpenAI-compatible abstraction across Gemini/Mistral/OpenAI/OpenRouter/Ollama— a new vendor is one catalog row.
- Explicit per-model base-URL + key routing (fixed the real bug where an OpenAI model inherited the Gemini base URL).
- Parameter adaptation makes strict APIs work without caller changes; never silently empty (402 shrink-and-retry, real errors surfaced).
- Clean separation: base provider stays simple/stateless; `retry/fallback` complexity isolated in `ResilientProvider`.
- Lazy imports + cheap `is_available` keep startup light and degrade gracefully when a key/dep is missing.

How to improve

- Classify errors by typed exception (`RateLimitError`, `APITimeoutError`, ...) not lowercased substring matching ('rate','429') — locale-proof and robust.
- Honor server backoff hints (`Retry-After` / `x-ratelimit-reset`) with jitter instead of fixed 1->2->4s.
- Wrap the live CHAT path in a resilient provider too — today a single 429 on the chat model fails the user turn with no fallback (only the agent path is protected).
- Stream live in `ResilientProvider` (track first-token emission) instead of full-buffering, which defeats real-time streaming.
- Move `ResilientProvider` into `backend/llm/` as a reusable layer and unit-test the backoff/fallback path.
- Set explicit client timeout/max-retries; name the magic numbers (4 iters, floor 64, 8s cap) per the coding-style rule.

4g · Web App / API / Streaming / Auth / Frontend (webapp)

What it does

The FastAPI web layer and vanilla-JS workspace users interact with. It serves the chat UI, streams answers + agent/ingest progress as newline-delimited JSON, manages per-user sessions with a signed cookie, handles login/signup/reset/Google OAuth, exposes CRUD for conversations and the ChatGPT-style version tree, drives PDF upload + live ingestion, and lets the user switch the active LLM. The frontend renders markdown + math + clickable citations, a reasoning timeline, a coding-agent step timeline with an in-card IDE/console, a sources drawer, and a Three.js background.

How it works

require_login (global) -> per-IP rate check -> _require_owner ownership check -> dispatch. Chat uses a sync generator (stream_chat_events) run in a threadpool, yielding event dicts as NDJSON; the agent/research routes use a worker thread pushing events onto a queue the response drains. The frontend is one IIFE: throttled rAF markdown rendering, citation chips resolved by source number (.n), a lazy-loaded version tree, a 401->login redirect, and a Three.js neural background with a Canvas-2D fallback.

Strengths

- Centralized security gate: a global require_login + _require_owner enforced BEFORE streaming, so no endpoint can leak another user's conversation.
- Hardened auth: anti-enumeration forgot-password with timing equalization, Host-header-poisoning defense, OAuth state nonce, reset-token validated before use.
- Clean NDJSON streaming contract with a documented event vocabulary; the right producer pattern for chat (sync gen) vs agent (queue+worker).
- Dependency-light but polished UX: throttled rendering, citation chips, version tree, graceful 401, Three.js background with guaranteed fallback.
- Operational care: mtime cache-busting + no-store, model switch rewrites endpoint+key+model together, optional systems degrade gracefully.

How to improve

- Add CSRF protection (Origin allowlist or double-submit token) on the JSON POSTs — SameSite=Lax alone isn't enough for multi-user deployment.
- Prune the in-process rate-limiter dict (one deque per (endpoint, IP), never evicted) or move to Redis with TTL — it's an unbounded slow leak.
- Gate /api/model behind an admin role — any logged-in user can currently change the global model/key for everyone and mutate .env on disk.
- Drive the Secure cookie flag from env (default on for https) instead of hardcoded https_only=False.
- Wire or remove the dead /api/research + preview.html (the deep-research agent is unreachable from the live UI).
- Split app.js (~1400-line IIFE) into ES modules and add Pydantic request models for typed validation at the boundary.

5 · Recent Enhancements & Improvement Roadmap

Recent enhancements (this work-stream):

- Conversation-aware routing: follow-ups answered from the chat / by re-running prior code, not a fresh web sweep (confidence floor + deixis veto so new questions still search).
- Latency: agentic verify-rewrite early-stop + DEEP_MAX_LOOPS cap, cross-loop evidence cache, concurrent follow-up search, per-stage timing — deep thoroughness preserved.
- Freshness & accuracy: live current-date anchor, freshness queries go web-only + current-year, entity-attribution guard, cited-only source panel.
- Code agent: output discipline (head+tail capture so requested values are never truncated) and freeze-verified-parts refinement (keep what passes, fix only failures; honest stall/partial).
- UI: light/dark code blocks, calmer background, scrollable agent/reason timelines, single-result agent output.

Highest-leverage improvements (cross-cutting):

- SCALE retrieval: replace the $O(N)$ per-query BM25 scan with an inverted index / Oracle Text, and invalidate caches on new ingestion.
- SAFETY for multi-tenant: CSRF + admin-gated model switch + Secure cookie + opt-in SSRF guard + bounded rate-limiter.
- RESILIENCE: wrap the live chat path in the resilient provider (retry/fallback), typed error classification, honor Retry-After.
- INGESTION: write native VECTOR directly (drop a stage), carry real page numbers through Docling, purge deleted PDFs.
- CONCURRENCY-SAFETY: make the fast/deep mode profile request-scoped instead of mutating global `os.environ`.
- AGENT RIGOR: cross-provider oracle/definition checks, value-adjacent completeness, early-exit on first verified candidate.
- MAINTAINABILITY: split `stream_chat_events` and `app.js` into focused modules per the project's own size rules.